STM32G431 • TM1934 • RGB LED

TM1934 学习笔记

单线三通道 RGB 驱动原理与函数详解

工程：C:/stm32lezi/TM1934 | 整理日期：2026-10-27 | 主控：STM32G431VBTx

基于 STM32G431 的单线 RGB LED 驱动实践

- 工程路径：`C:\stm32lezi\TM1934`
- 关键引脚：`PA0` → 74HC245 → TM1934 `DIN`，`PF2` → 74HC245 `DIR`
- 驱动文件：`Core/Src/tm1934.c`、`Core/Inc/tm1934.h`
- 整理日期：2026-10-27

---

1. 学习目标

这份笔记围绕本工程中的 TM1934 驱动代码展开，重点回答三个问题：

1. TM1934 的单线协议到底怎样表示 0、1、复位和数据帧。
2. `tm1934.c` 中每个函数为什么这样写，原理是什么。
3. 在实际工程中应该怎样调用这些函数，如何修改级联数量、颜色和动画。

学完后应能独立完成：

- 用 GPIO 模拟 TM1934 时序；
- 点亮任意一颗级联 LED；
- 修改 RGB 颜色；
- 编写渐变、流水灯等动画；
- 根据级联数量估算 RAM 占用和刷新时间；
- 定位“灯不亮、颜色错、闪烁、只亮第一颗”等常见问题。

---

2. TM1934 关键特性

项目	参数 / 说明
类型	单线三通道 RGB 恒流驱动
通道	OUTR、OUTG、OUTB
灰度	每通道 8 bit，256 级 PWM
单颗数据量	24 bit = R[7:0] + G[7:0] + B[7:0]
数据速率	典型 800 kHz，周期 1.25 μs
位编码	归 0 码，高电平宽度区分 0 / 1
级联	`DO` 转发后级数据
输出耐压	最高 24 V
恒流	约 14～15 mA（以具体批次手册为准）
PWM 频率	约 666 Hz
复位	低电平大于 200 μs
数据顺序	高位先发，R → G → B

> 注意：TM1934 与 WS2812 的“0/1 码”高电平宽度不同，不能直接套用 WS2812 的时序参数。本文参数来自工程目录中的 `TM1934_V1.4.pdf`。

---

3. 硬件连接与方向控制

3.1 信号路径

本工程中 PA0 不是直接连到 TM1934，而是经过一片 74HC245 缓冲器：

```
STM32 PA0  ⟶  74HC245 A1  ⟶  B1  ⟶  J14.1 (LED1)  ⟶  TM1934 DIN
STM32 PF2  ⟶  74HC245 DIR
```

74HC245 的 `DIR` 决定数据传输方向：

- `DIR = 1`：A → B，MCU 输出到 LED 板；
- `DIR = 0`：B → A，方向相反。

因此只配置 PA0 还不够，**PF2 必须输出高电平**，否则 PA0 的数据无法到达 TM1934。驱动中的 `TM1934_Init()` 已经完成这一步。

3.2 数据线空闲电平

TM1934 的数据线在空闲时应保持低电平。驱动初始化时执行：

```
HAL_GPIO_WritePin(TM1934_GPIO_PORT, TM1934_GPIO_PIN, GPIO_PIN_RESET);
```

这样在发送第一帧之前不会产生错误的起始沿。

3.3 电源与地

- MCU、74HC245、TM1934 必须共地。
- TM1934 的 VDD 与 LED 供电按灯板设计处理。
- 长线、多个灯板级联时，建议在 DIN 上串联小电阻并做好电源去耦。
- 若灯板使用 5 V 供电而 MCU 为 3.3 V，要确认 TM1934 的输入高电平阈值是否满足；必要时增加电平转换。

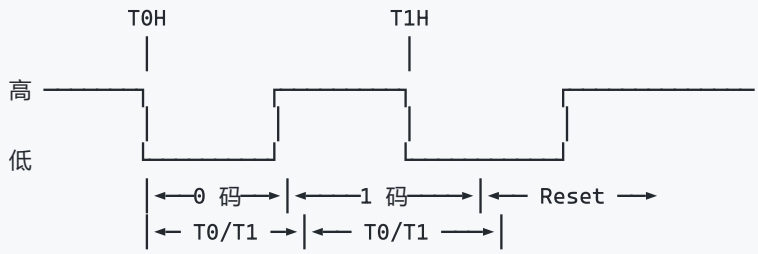
---

4. 单线协议原理

4.1 位编码：高电平宽度决定 0 或 1

TM1934 每个 bit 的周期约为 1.25 μs，通过高电平持续时间区分：

参数	含义	最小值	典型值	最大值
T0H	0 码高电平时间	310 ns	360 ns	410 ns
T1H	1 码高电平时间	650 ns	720 ns	1000 ns
T0/T1	单个 bit 周期	1.25 μs	1.25 μs	2.5 μs
Treset	复位低电平时间	200 μs	—	20000 μs

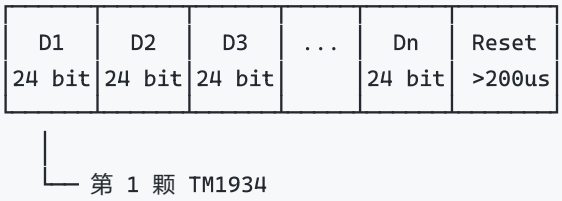


要点：

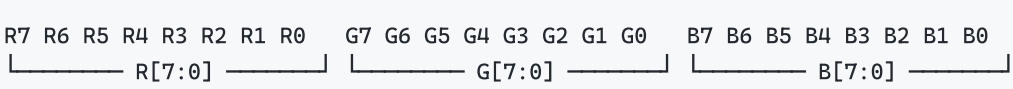
- 0 码：高电平窄，约 340 ns；
- 1 码：高电平宽，约 720 ns；
- 每个 bit 都有固定的低电平“间隔”；
- 连续发送时，字节之间低电平不能超过 50 μs，否则芯片可能误判为复位；
- 只有低电平持续大于 200 μs 才是一次真正的复位 / 锁存。

4.2 数据帧结构

一帧完整数据由若干 24 bit 显示数据和一个 Reset 组成：



每颗 TM1934 对应一个 24 bit 数据包，数据包内部顺序为：



- 每个字节都是 **MSB 先发**；
- 先发 R，再发 G，最后发 B；
- 全 0 表示关闭，全 1 表示该通道最大占空比；
- 驱动内部帧缓存按 `R, G, B` 连续存放，`TM1934_Show()` 按顺序逐字节发出。

4.3 级联与转发

- 第 1 颗芯片接收前 24 bit 后，开始把后续数据从 `DO` 转发给下一颗；
- 第 2 颗芯片收到第 2 组 24 bit，再转发第 3 组；
- 所有芯片收到数据后，控制器发送 Reset；
- 芯片在 Reset 后把各自保存的 24 bit 解码为 PWM 输出；
- 如果发送的数据包数量少于实际级联数量，后面的灯不会得到正确数据；
- 如果发送的数据包数量多于实际级联数量，多出的数据会从最后一颗芯片的 `DO` 输出，不影响前面的灯。

因此 `TM1934_LED_COUNT` 应与实际级联颗数一致，至少不要少于实际颗数。

---

5. 驱动代码结构

```
main.c
  ├─ LED1_ColorWheel()      颜色轮
  ├─ LED1_Gradient()        渐变 / 呼吸
  └─ LED1_Running()         流水灯
            │
            ▼
tm1934.c
  ├─ TM1934_SetPixel()      写帧缓存
  ├─ TM1934_SetAll()        全部设置为同一颜色
  ├─ TM1934_Clear()         清空帧缓存
  └─ TM1934_Show()          发送整帧
            │
            ▼
  ├─ TM1934_DelayUs()       微秒延时
  ├─ TM1934_WaitUntil()     绝对周期等待
  ├─ TM1934_WriteBit()      发送 1 bit
  └─ TM1934_WriteByte()     发送 1 byte (MSB 先发)
```

这种分层方式的核心思想是：**应用层只改颜色，驱动层统一刷新。**

---

6. 函数原理与使用方法（重点）

6.1 `TM1934_Init()`

作用

完成 GPIO、方向控制、DWT 计时器和时序参数的初始化，并发送一帧全 0 数据。

关键步骤

1. 使能 GPIOA、GPIOF 时钟；
2. 配置 PF2 为推挽输出并输出高电平，保证 74HC245 的 A → B 方向；
3. 配置 PA0 为推挽输出并输出低电平；
4. 使能 DWT 的周期计数器 `CYCCNT`；
5. 根据 `SystemCoreClock` 计算 T0H、T1H、周期对应的 CPU 周期数；
6. 清空帧缓存并发送一帧全 0，确保上电后灯是熄灭的。

DWT 计时原理

```
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0U;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
```

`DWT->CYCCNT` 每个 CPU 时钟加 1。系统主频为 170 MHz 时：

```
1 个周期 = 1 / 170 MHz ≈ 5.88 ns
340 ns ≈ 58 个周期
720 ns ≈ 122 个周期
1.25 µs ≈ 213 个周期
```

代码中的计算方式是：

```
tm1934_t0h_cycles = (SystemCoreClock / 1000U) * TM1934_T0H_NS / 1000000U;
```

先除以 1000，再乘纳秒数，可以在 32 位整数范围内完成计算：

```
170000 × 340 / 1000000 = 57.8 → 57 个周期
```

调用位置

必须在 `SystemClock_Config()` 之后调用，因为时序周期数依赖最终主频：

```
HAL_Init();
SystemClock_Config();
MX_GPIO_Init();
TM1934_Init();
```

注意事项

- `TM1934_Init()` 会自己配置 PA0 和 PF2，不依赖 CubeMX 的引脚配置；
- 如果更换引脚，需要修改 `tm1934.c` 顶部的 `TM1934_GPIO_PORT`、`TM1934_GPIO_PIN`、`TM1934_DIR_PORT`、`TM1934_DIR_PIN`；
- 如果主频改变，周期数会在运行时自动重新计算。

6.2 `TM1934_SetPixel()`

原型

```
void TM1934_SetPixel(uint16_t index, uint8_t red, uint8_t green, uint8_t blue);
```

作用

把指定编号的 LED 颜色写入帧缓存。此函数**不会立即输出**，必须再调用 `TM1934_Show()`。

原理

帧缓存是一个连续数组：

```
static uint8_t tm1934_buffer[TM1934_LED_COUNT * 3U];
```

第 `index` 颗 LED 的偏移量为：

```
uint16_t offset = index * 3U;
tm1934_buffer[offset]     = red;
tm1934_buffer[offset + 1] = green;
tm1934_buffer[offset + 2] = blue;
```

例如第 2 颗 LED（下标从 0 开始）：

```
index = 2
offset = 6
buffer[6] = R, buffer[7] = G, buffer[8] = B
```

使用方法

```
TM1934_SetPixel(0, 255, 0, 0);   // 第 1 颗红色
TM1934_SetPixel(1, 0, 255, 0);   // 第 2 颗绿色
TM1934_SetPixel(2, 0, 0, 255);   // 第 3 颗蓝色
TM1934_Show();                   // 统一刷新
```

为什么要“先设置、后刷新”

- 可以一次性修改整条灯带，避免看到“改一颗亮一颗”的中间状态；
- 动画计算和实际发送分离，代码更清晰；
- 刷新时关闭中断，整帧数据连续发送，减少闪烁和错位。

边界检查

```
if (index >= TM1934_LED_COUNT)
{
  return;
}
```

越界写入会破坏其他内存，因此必须保留。

6.3 `TM1934_SetAll()`

作用

把整条灯带设置为同一种颜色。

原理

循环调用 `TM1934_SetPixel()`：

```
for (uint16_t i = 0U; i < TM1934_LED_COUNT; i++)
{
  TM1934_SetPixel(i, red, green, blue);
}
```

使用方法

```
TM1934_SetAll(0, 255, 0);
TM1934_Show();          // 全部绿色
```

适合做整条灯带的呼吸、闪烁、单色测试。

6.4 `TM1934_Clear()`

作用

把帧缓存全部清零，即所有 LED 关闭。

原理

```
for (uint16_t i = 0U; i < sizeof(tm1934_buffer); i++)
{
  tm1934_buffer[i] = 0U;
}
```

注意：清空帧缓存后，灯不会立刻熄灭，仍需调用 `TM1934_Show()`。

典型用法

流水灯每一帧先清空，再点亮“头部”和“拖尾”：

```
TM1934_Clear();
TM1934_SetPixel(head, 255, 255, 255);
TM1934_Show();
```

6.5 `TM1934_Show()`

作用

把帧缓存中的整帧数据发送给 TM1934，并产生锁存复位。

完整流程

```
保存 PRIMASK
  ↓
关闭全局中断
  ↓
数据线拉低 300 μs          ← 起始复位，让芯片进入接收状态
  ↓
逐字节发送帧缓存           ← 每颗 LED 24 bit，MSB 先发
  ↓
数据线拉低 300 μs          ← 结束复位，芯片锁存并输出 PWM
  ↓
恢复 PRIMASK
```

为什么前后都要复位

- 前一次复位：确保芯片从“上一帧”状态回到接收准备状态，避免帧边界错乱；
- 后一次复位：TM1934 只有在检测到 Reset 后才把收到的 24 bit 数据解码输出；
- 两次复位都使用 300 μs，处于手册要求的 200 μs 以上范围。

为什么关闭中断

TM1934 一个 bit 只有 1.25 μs。如果发送过程中进入 SysTick 或串口中断，可能把某个高电平拉长到几微秒，芯片就会把 0 码误判为 1 码，表现为颜色错乱、闪烁或整条灯带异常。

因此 `TM1934_Show()` 使用临界区：

```
uint32_t primask = __get_PRIMASK();
__disable_irq();

/* 发送整帧 */

__set_PRIMASK(primask);
```

使用 `PRIMASK` 保存和恢复，而不是直接 `__enable_irq()`，可以避免破坏调用者原有的中断状态。

临界区时间估算

以 8 颗 LED 为例：

```
数据时间 ≈ 8 × 24 × 1.25 µs = 240 µs
两次复位 ≈ 600 µs
总临界区 ≈ 840 µs
```

如果每 20 ms 刷新一次，则 CPU 被占用约：

```
840 µs / 20 ms ≈ 4.2%
```

对于 8 颗 LED 完全可以接受。但如果级联几百颗，临界区会迅速变长，这时应考虑 SPI + DMA 或 PWM + DMA 方案。

使用方法

```
TM1934_SetAll(255, 0, 0);
TM1934_Show();
```

注意

- 不要在中断里调用 `TM1934_Show()`，它不是可重入函数；
- 不要在主循环和中断中同时操作帧缓存；
- 刷新频率不需要太高，20～50 Hz 即可满足动画需求。

6.6 内部时序函数

6.6.1 `TM1934_DelayUs()`

作用

基于 DWT 的微秒级忙等待，主要用于 300 µs 复位。

原理

```
uint32_t start = DWT->CYCCNT;
uint32_t cycles = us * tm1934_cycles_per_us;

while ((DWT->CYCCNT - start) < cycles)
{
}
```

- `tm1934_cycles_per_us = SystemCoreClock / 1000000`；
- 170 MHz 时，每微秒 170 个周期；
- 无符号减法在计数器回绕时仍然正确；
- 不使用 `HAL_Delay()`，因为 `HAL_Delay()` 依赖 SysTick 中断，而 `TM1934_Show()` 中已经关闭了中断。

6.6.2 `TM1934_WaitUntil()`

作用

等待 DWT 计数器到达一个绝对目标值。

原理

```
while ((int32_t)(DWT->CYCCNT - target) < 0)
{
}
```

- 使用有符号差值判断是否到达目标；
- 计数器回绕时仍能正确工作；
- 采用“绝对时间点”而不是“相对延时”，可以避免每个 bit 的误差累积。

为什么用绝对时间

如果每个 bit 都写成“延时 340 ns”，函数调用、循环和 Flash 取指的开销会不断累加，最终周期越来越长。使用绝对目标：

```
uint32_t start = DWT->CYCCNT;
/* 拉高 */
TM1934_WaitUntil(start + high_cycles);
/* 拉低 */
TM1934_WaitUntil(start + period_cycles);
```

每个 bit 都以自己的 `start` 为基准，误差不会跨 bit 累积。

6.6.3 `TM1934_WriteBit()`

作用

发送一个 TM1934 bit。

原理

```
uint32_t start = DWT->CYCCNT;

TM1934_GPIO_PORT->BSRR = TM1934_GPIO_PIN;

TM1934_WaitUntil(start + (value ? tm1934_t1h_cycles : tm1934_t0h_cycles));

TM1934_GPIO_PORT->BSRR = (uint32_t)TM1934_GPIO_PIN << 16U;

TM1934_WaitUntil(start + tm1934_period_cycles);
```

对应关系：

- `value = 1`：等待 `T1H`，高电平约 720 ns；
- `value = 0`：等待 `T0H`，高电平约 340 ns；
- 高电平结束后拉低，并等待到完整周期结束。

为什么用 BSRR

`BSRR` 是“置位 / 复位寄存器”，写 1 到低 16 位会置高，写 1 到高 16 位会拉低，不需要读-改-写，速度快且不会被中断打断后破坏其他引脚：

```
GPIOA->BSRR = GPIO_PIN_0;              // PA0 = 1
GPIOA->BSRR = (uint32_t)GPIO_PIN_0 << 16U;  // PA0 = 0
```

6.6.4 `TM1934_WriteByte()`

作用

发送一个字节，高位先发。

原理

```
for (uint8_t mask = 0x80U; mask != 0U; mask >>= 1U)
{
  TM1934_WriteBit((value & mask) != 0U);
}
```

发送顺序：

```
bit7 → bit6 → bit5 → bit4 → bit3 → bit2 → bit1 → bit0
```

TM1934 要求每个颜色字节都是 MSB 先发，因此这里必须从 `0x80` 开始。

6.7 应用层动画函数

6.7.1 `LED1_ColorWheel()`

作用

把一个 0～255 的位置映射成彩虹颜色，并按亮度缩放。

原理

把色环分成三段：

```
0   ~ 84   : 红 → 黄 → 绿
85  ~ 169  : 绿 → 青 → 蓝
170 ~ 255  : 蓝 → 紫 → 红
```

每段使用线性插值：

```
if (position < 85U)
{
  r = 255U - position * 3U;
  g = position * 3U;
  b = 0U;
}
```

最后按亮度缩放：

```
*red = (uint8_t)((uint16_t)r * brightness / 255U);
```

这里先转成 `uint16_t`，避免 `r * brightness` 超过 255 后溢出。

使用方法

```
uint8_t r, g, b;
LED1_ColorWheel(0, 255, &r, &g, &b);     // 红色
LED1_ColorWheel(85, 255, &r, &g, &b);   // 绿色
LED1_ColorWheel(170, 255, &r, &g, &b);  // 蓝色
```

6.7.2 `LED1_Gradient()`

作用

实现“渐变 + 呼吸”效果：

- 空间上：整条灯带的颜色按位置错开，形成彩虹渐变；
- 时间上：整体亮度按三角形波从 0 到 255 再回到 0。

亮度呼吸

```
uint32_t brightness = phase % 512U;
if (brightness > 255U)
{
  brightness = 511U - brightness;
}
```

`phase` 每 20 ms 增加 1，因此：

```
0 → 255: 约 5.1 秒
255 → 0: 约 5.1 秒
完整呼吸周期: 约 10.2 秒
```

空间渐变

```
uint8_t position = phase + i * 256U / TM1934_LED_COUNT;
```

第 `i` 颗 LED 的颜色位置比前一颗偏移 `256 / N`。例如 8 颗 LED 时，每颗偏移 32，整条灯带正好铺满一个色环。

调用方式

```
LED1_Gradient(HAL_GetTick());
HAL_Delay(20);
```

6.7.3 `LED1_Running()`

作用

实现带拖尾的流水灯。

原理

- `head` 每 80 ms 前进一步；

- 头部最亮；
- 后面 3 颗作为拖尾，亮度依次为 `255 >> 1`、`255 >> 2`、`255 >> 3`；
- 每一帧先 `TM1934_Clear()`，再绘制头部和拖尾；
- 颜色位置随时间和 LED 下标变化，因此流水过程中颜色也会流动。

头部计算

```
uint32_t head = (tick / LED1_CHASE_STEP_MS) % (TM1934_LED_COUNT + LED1_CHASE_TAIL);
```

`+ LED1_CHASE_TAIL` 让头部在离开灯带后继续“走”几帧，形成拖尾逐渐消失的效果。

单颗 LED 的特殊处理

如果 `TM1934_LED_COUNT == 1`，空间流水没有意义，因此改为单颗 LED 的颜色循环：

```
LED1_ColorWheel((uint8_t)(tick / 8U), 255U, &r, &g, &b);
TM1934_SetPixel(0U, r, g, b);
TM1934_Show();
```

这样即使只焊了一颗 TM1934，也能看到“流水”颜色变化。

---

7. 常用示例

7.1 点亮第一颗 LED

```
TM1934_Init();

TM1934_SetPixel(0, 255, 0, 0);
TM1934_Show();

while (1)
{
}
```

7.2 全部点亮为白色

```
TM1934_SetAll(255, 255, 255);
TM1934_Show();
```

7.3 全部关闭

```
TM1934_Clear();
TM1934_Show();
```

7.4 自定义动画框架

```
while (1)
{
  uint32_t now = HAL_GetTick();

  for (uint16_t i = 0; i < TM1934_LED_COUNT; i++)
  {
    uint8_t level = (uint8_t)((now / 10U + i * 16U) & 0xFFU);
    TM1934_SetPixel(i, level, 0, 255U - level);
  }

  TM1934_Show();
  HAL_Delay(20);
}
```

7.5 修改级联数量

在 `tm1934.h` 中修改：

```
#define TM1934_LED_COUNT 8U
```

RAM 占用约为：

```
3 字节/颗 × 级联数量
```

例如：

级联数量	帧缓存 RAM	单帧数据时间（约）
1	3 B	30 µs
8	24 B	240 µs
64	192 B	1.92 ms
256	768 B	7.68 ms
1000	3000 B	30 ms

---

8. 时序与性能计算

8.1 周期数计算

系统主频 170 MHz：

项目	目标时间	计算	周期数
T0H	340 ns	170000 × 340 / 1000000	57
T1H	720 ns	170000 × 720 / 1000000	122
周期	1250 ns	170000 × 1250 / 1000000	212
复位	300 µs	300 × 170	51000

由于函数调用、`BSRR` 写入和等待循环本身有少量开销，实际高电平会比目标值多出几个周期。当前参数把 T0H 目标设为 340 ns，而不是手册典型值 360 ns，就是为了给这部分开销留出余量，使实际高电平仍落在 310～410 ns 范围内。

8.2 刷新率

单颗 TM1934 传输时间：

```
24 bit × 1.25 µs = 30 µs
```

1000 颗灯带：

```
1000 × 30 µs = 30 ms
刷新率 ≈ 33 Hz
```

实际代码中每帧还有两次 300 µs 复位，并且 bit 周期略大于 1.25 µs，因此实际刷新率会略低。

8.3 当前实现的速度限制

`TM1934_Show()` 在整帧发送期间关闭中断。对于 8 颗 LED，临界区约 0.9 ms，没有问题；但对于几百颗 LED，会出现：

- 中断响应延迟；
- SysTick 丢 tick；
- 串口 / DMA 回调延迟；
- 系统实时性下降。

如果以后要驱动长灯带，建议改成：

1. **SPI + DMA**：用 SPI 的 4 个 bit 表示 TM1934 的 1 个 bit；
2. **PWM + DMA**：用定时器 DMA 输出占空比序列；
3. 或者至少把临界区缩短到每个字节，而不是整帧。

---

9. 常见问题排查

9.1 灯完全不亮

依次检查：

1. PF2 是否为高电平，74HC245 方向是否正确；
2. J14 的 LED1 是否接到 TM1934 的 DIN；
3. TM1934 电源和地是否正常；
4. 数据线空闲时是否为低电平；
5. `TM1934_LED_COUNT` 是否与实际级联数量匹配；
6. 是否调用了 `TM1934_Show()`，只调用 `TM1934_SetPixel()` 不会输出。

9.2 颜色错乱

- 确认数据顺序是 R → G → B；
- 如果灯板实际是 G → R → B，需要在 `TM1934_SetPixel()` 或 `TM1934_Show()` 中调整顺序；
- 检查是否把 0 码和 1 码的高电平时间写反；
- 检查发送过程中是否被中断打断。

9.3 闪烁或随机错位

- 发送整帧时必须关闭中断；
- 数据线过长时检查信号完整性；
- 确保 MCU、缓冲器、灯板共地；
- 电源功率不足也会导致颜色异常或重启。

9.4 只有第一颗亮

- 实际只焊了一颗 TM1934；
- 级联数据线 `DO → DIN` 没有接好；
- `TM1934_LED_COUNT` 小于实际颗数；
- 数据包数量不足，后面的芯片没有收到完整 24 bit。

9.5 亮度偏低或偏色

- TM1934 是恒流驱动，亮度主要由 PWM 占空比决定；
- 检查供电电压和灯板串接电阻；
- 8 bit 灰度只有 256 级，低亮度时可能出现轻微跳变；
- 可以在应用层做 gamma 校正改善视觉效果。

---

10. 关键结论

1. TM1934 用高电平宽度区分 0 / 1，典型 T0H = 360 ns，T1H = 720 ns，周期 1.25 µs。
2. 一帧数据由 `D1 D2 ... Dn` 加一个大于 200 µs 的低电平 Reset 组成。
3. 每颗芯片 24 bit，顺序是 R → G → B，每个字节 MSB 先发。
4. PA0 只负责数据，74HC245 的 PF2 方向控制必须为高。
5. `SetPixel / SetAll / Clear` 只改帧缓存，`Show` 才真正发送。

6. `Show` 必须在临界区内完成，否则一个中断就可能导致整帧错位。
7. 当前驱动适合几十颗以内的级联；更长灯带应改用 SPI/PWM + DMA。
8. 修改级联数量只需要改 `TM1934_LED_COUNT`，但要注意 RAM 和刷新时间。

附录 A：API 速查

函数	作用	是否立即输出
`TM1934_Init()`	初始化 GPIO、DWT、时序参数	会发送一帧全 0
`TM1934_SetPixel(i, r, g, b)`	设置第 i 颗颜色	否
`TM1934_SetAll(r, g, b)`	设置全部颜色	否
`TM1934_Clear()`	清空帧缓存	否
`TM1934_Show()`	发送整帧并锁存	是

附录 B：推荐调用顺序

```
HAL_Init();
SystemClock_Config();
MX_GPIO_Init();

TM1934_Init();

TM1934_SetAll(0, 0, 0);
TM1934_Show();

while (1)
{
  /* 修改帧缓存 */
  /* TM1934_Show() */
  /* HAL_Delay(...) */
}
```